
Algorithm 11: Bounded Backward Half-Search

Input:

- V . Set of nodes
- $E_{s,in}(v)$. Set of scheduled INBOUND edges to v : $(w, v, \text{dep}, \text{arr})$
- $E_{t,in}(v)$. Set of INBOUND transfer edges to v : (w, v, Δ)
- $\text{country}(v)$. Country associated with node v
- $R_{\text{back}}(v, b)$. Precomputed reverse reachable country sets
- $MTT(c)$. Precomputed minimum transit times
- $\text{bucket}(t)$. Time bucket mapping function
- MaxDuration . Maximum allowed elapsed time for the half-search (e.g., 840 mins)
- MinCountries . Minimum unique countries required to save the path (e.g., 6)
- GlobalMinTime . Minimum realistic time to visit a country (e.g., 25 mins)

Output:

- Labels_B . Map of nodes to a set of Pareto-optimal backward labels

```

1: Initialize  $\text{Labels}_B(v) \leftarrow \emptyset$  for all  $v \in V$ 
2: Initialize priority queue  $Q \leftarrow \emptyset$  // ordered by decreasing score
3: // Initialize labels at arrivals (End of the journey)
4: for all  $v \in V$  do
5:   for all scheduled edges  $(w, v, \text{dep}, \text{arr}) \in E_{s,in}(v)$  do
6:     // We initialize the state sitting at the arrival node 'v', having just finished our journey
7:      $L \leftarrow (\text{node} = v, t = \text{arr}, C = \{\text{country}(v)\}, \text{succ} = \text{null}, \text{end\_time} = \text{arr})$ 
8:      $\text{Labels}_B(v) \leftarrow \text{Labels}_B(v) \cup \{L\}$ 
9:     // Score favors paths with MORE countries and LATER departure times (less elapsed time)
10:     $\text{compute\_score}(L) \leftarrow (|L.C| \times 10000) + L.t$ 
11:     $Q \leftarrow Q \cup \{L\}$ 
12: // Main loop
13: while  $Q \neq \emptyset$  do
14:   extract  $L = (v, t, C, \text{succ}, \text{end\_time})$  with maximum score from  $Q$ 
15:    $\text{elapsed} \leftarrow \text{end\_time} - t$ 
16:    $\text{remaining\_half\_time} \leftarrow \text{MaxDuration} - \text{elapsed}$ 
17:   // — 1. GLOBAL HARD FLOOR PRUNING (Backward Variant) —
18:    $\text{needed} \leftarrow \text{MinCountries} - |C|$ 
19:   if  $\text{needed} > 0 \wedge \text{remaining\_half\_time} < (\text{needed} \times \text{GlobalMinTime})$  then
20:     continue
21:   // — 2. TIME-BUDGETED UB PRUNING (Backward Variant) —
22:   if  $\text{needed} > 0$  then
23:      $b \leftarrow \text{bucket}(t)$ 
24:      $U \leftarrow R_{\text{back}}(v, b) \setminus C$ 
25:     Sort  $U$  as a sequence  $(c_1, c_2, \dots, c_k)$  such that  $MTT(c_1) \leq MTT(c_2) \leq \dots \leq MTT(c_k)$ 
26:      $\text{time\_spent} \leftarrow 0$ 
27:      $\text{UB} \leftarrow 0$ 
28:     for  $i = 1$  to  $|U|$  do
29:        $\text{time\_spent} \leftarrow \text{time\_spent} + MTT(c_i)$ 
30:       if  $\text{time\_spent} \leq \text{remaining\_half\_time}$  then
31:          $\text{UB} \leftarrow \text{UB} + 1$ 
32:       else

```

Algorithm 11 is essentially the same as Algorithm 10, but it runs backwards in time. The main difference is that we had to change the maths of the graph (check for incoming edges instead of outgoing, modify pareto dominance rules, etc.). Other than that, the logic is the same.

Algorithm 11 is essentially the same as Algorithm 10, but it runs backwards in time. The main difference is that we had to change the maths of the graph (check for incoming edges instead of outgoing, modify pareto dominance rules, etc.). Other than that, the logic is the same.